EECS 348: Software Engineering I

EECS 348 Group 5
Software Requirements Specifications
Version <1.0>

Revision History

Date	Version	Description	Author
3/12/2026	1.5	Completed Section 4	Alec
3.13/2026	1.6	Completed Section 2	Lea
3/10/2026	1.7	Completed Section 1	Alex
3/13/2026	1.8	Completed Section 3	Jr
3/13/2026	1.9	Completed Section 3	Zack
3/13/2026	2.0	Completed Section 5	Shashwat

Table of Contents

1. Introduction 4
   - 1.1 Purpose 4
   - 1.2 Scope 4
   - 1.3 Definitions, Acronyms, and Abbreviations 4
   - 1.4 References 4
   - 1.5 Overview 4

2. Overall Description 5
   - 2.1 Product perspective 5
     - 2.1.1 System Interfaces 5
     - 2.1.2 User Interfaces 5
     - 2.1.3 Hardware Interfaces 5
     - 2.1.4 Software Interfaces 5
     - 2.1.5 Communication Interfaces 5
     - 2.1.6 Memory Constraints 5
     - 2.1.7 Operations 5
   - 2.2 Product functions 5
   - 2.3 User characteristics 5
   - 2.4 Constraints 5
   - 2.5 Assumptions and dependencies 5
   - 2.6 Requirements subsets 5

3. Specific Requirements 5
   - 3.1 Functionality 5
   - 3.2 Use-Case Specifications 6
   - 3.3 Supplementary Requirements 6

4. Classification of Functional Requirements 6

5. Appendices 6

Software Requirements Specifications

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to provide a comprehensive description of the Arithmetic Expression Evaluator. This document specifies the requirements for a C++ implementation that can parse string-based input, handle mathematical operators with correct precedence, and evaluate complex expressions containing nested parentheses.

1.2 Scope

The Arithmetic Expression Evaluator is a C++ application that translates user-entered text strings into a mathematical result. To do this, the program accepts a string from the user and breaks it into components, classifying characters as integers, operators, or parentheses.

The program supports the following operators and maps them with the same internal logic structure as PEMDAS:

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)
- Exponentiation (**)
- Modulo (%)

The evaluator recognizes negative signs and uses parentheses to correctly group and solve the expression in the right order.

1.3 Definitions, Acronyms, and Abbreviations

- SRS: Software Requirements Specification: This document describes the project.
- PEMDAS: The standard order of operations used to solve expressions.
- Tokenization: Breaking down the user's input string into smaller components, which allows for easier programming and processing of the math expression.
- Operators: Characters like +, -, *, /, %, and ** that tell the program which math operation to perform.
- Integers: A whole number used for the initial version of this math program.
- Negative sign: a minus symbol used in front of a number to show it is a negative value.

1.4 References

- EECS 348 Project 2 Assignment Description: “Arithmetic Expression Evaluator” project instructions and requirements provided by Professor Hosein Saiedian
- Project Plan: Arithmetic Expression Evaluator: Internal document outlining the development timeline, task breakdown, and milestones for the project.
- EECS 348 SRS Template Guide: The guide provided in class showed how this document should be organized and what sections to include.

1.5 Overview

This SRS describes the complete software requirements for the system. It includes the purpose, scope, definitions, acronyms, abbreviations, references, and overview of the SRS. The document is organized into the following sections:

- Section 2 (Overall Description) describes the general factors that affect the product and its requirements, including product perspective, functions, user characteristics, and constraints.
- Section 3 (Specific Requirements) contains all the software requirements to a level of detail sufficient to enable designers to design a system to satisfy those requirements. This includes functionality, use-case specifications, and supplementary requirements.
- Section 4 (Classification of Functional Requirements) lists all functional requirements and orders

them by type, such as essential, desirable, and optional.

- Section 5 (Appendices) includes any additional information to be considered part of the requirements

2. Overall Description

2.1 Product perspective

The Arithmetic Expression Evaluator is developed in C++ as the expression parser component for a larger compiler product for language L. The program will utilize object-oriented programming principles to structure the code.

2.1.1 System Interfaces

The system will operate as a C++ program capable of parsing and evaluating arithmetic expressions.

2.1.2 User Interfaces

The program will feature a user-friendly and clear command-line interface. This interface allows users to enter expressions and displays the calculated results.

2.1.3 Hardware Interfaces

No specific hardware interfaces are required for this application.

2.1.4 Software Interfaces

This application will be developed using C++.

2.1.5 Communication Interfaces

No external communication or networking interfaces are required for this project.

2.1.6 Memory Constraints

The system will utilize internal data structures, like a stack or a tree, to represent the expression's structure in memory during evaluation.

2.1.7 Operations

The main operation is taking an arithmetic expression as input, parsing the text, and calculating the final mathematical result.

2.2 Product functions

The system will perform these primary functions:

- Tokenize the input expression the user has entered.
- Parse and evaluate arithmetic expressions that include numeric constants and the operators +, -, *, /, %, and **.
- Calculate the result according to PEMDAS.
- Apply left-to-right associativity for +, -, *, /, and %.
- Apply right-to-left associativity for **.
- Identify and evaluate expressions enclosed within parenthesis to properly determine the order of evaluation.
- Support the use of unary + and - operators.
- Manage error scenarios, such as division by zero or invalid expressions, through robust error handling.
- Report any errors that may have been encountered to the user.

2.3 User characteristics

- The user will interact with the system through the command-line interface.
- The user will provide input that consists of numeric constants.
- The user may input invalid expressions, like unmatched parenthesis, missing operands, or invalid characters.
- The user relies on the system to provide any errors whenever invalid input is detected.

2.4 Constraints

- The software must be developed using C++.
- The system's code structure must utilize object-oriented programming principles.
- The application interface is restricted to text-based, command-line interface.
- The project must follow the requirements provided in the project description and follow C++ implementation.
- The system must follow standard math rules, like division by zero is an invalid operation.

2.5 Assumptions and dependencies

- Assumptions:
  - All input entered by the user will contain integer numeric constants.
  - The program must be able to accept future changes that might be made later on.
- Dependencies:
  - The arithmetic expression evaluator is developed as a dependent component for a larger compiler product for a language called "L".

2.6 Requirements subsets

- Tokenization and Parsing: Requirements related to breaking down the input string and creating a data structure to represent the expression.
- Evaluation and Precedence: Requirements covering the implementation of PEMDAS, left-to-right associativity for standard operators, and right-to-left associativity for exponentiation.
- Grouping and Unary Operations: Identifies and evaluates nested parenthesis and handles unary positive and negative signs.
- Interface and Error Management: Requirements for command-line user interface and the robust error handling necessary to manage and report invalid expressions clearly.

3. Specific Requirements

The system to be developed is designed to evaluate arithmetic expressions by processing user-provided input and producing a correct numerical result. The system serves as the foundation for all functional and non-functional requirements described in this section.

3.1 Functionality

The system shall function as a versatile arithmetic expression evaluator. It shall accept a complete arithmetic expression as input, parse the expression, and evaluate it according to standard mathematical rules, including PEMDAS, operator precedence, associativity, and parentheses.

This system has the following arithmetic operators:

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)
- Exponentiation (**)
- Modulo (%)
- The system shall support unary operators, including the unary negative (-) and unary positive (+) signs, to indicate positive or negative values.
- The system shall recognize numeric constants within an expression and correctly evaluate them during computation.

3.1.1 <FR1: Expression Input>

The system shall cleanly take complete expression input and handle any user-syntax errors they might make when entering it.

3.1.2 *<FR2: Expression Parsing>*

The system shall parse the input expression into a valid internal representation before evaluation.

3.1.3 *<FR3: Operator Support>*

The system shall cleanly incorporate all previously specified operators.

3.1.4 *<FR4: Expression Precedence and Associativity>*

The system shall evaluate expressions according to standard mathematical precedence and associativity rules, including PEMDAS.

3.1.5 *<FR5: Parentheses Handling>*

The system shall correctly interpret and evaluate parentheses to override default operator precedence.

3.1.6 *<FR6: Unary Operator Handling>*

The system shall support unary positive (+) and unary negative (-) operators.

3.1.7 *<FR7: Recognizing Numeric Constants>*

The system shall recognize numeric constants within an expression and correctly calculate them during evaluation.

3.1.8 *<FR8: Display Result>*

The system shall display the final evaluated result to the user.

3.1.9 *<FR9: Error Detection and Recovery>*

The system shall detect invalid expressions and runtime errors, including division by zero and mismatched parentheses, and shall display a clear error message and allow the user to re-enter an expression after an error occurs.

3.2 Use-Case Specifications

- The system must be able to handle various arithmetic operators, including the following use cases:
  - Includes reading and handling unary positive and negative signs (+, -) as addition and subtraction cases.
  - Includes the ability to read a single asterisk (*) input as a multiplication marker and complete the corresponding calculation.
  - Includes the ability to read a double asterisk, notated as **, as an exponentiation marker and complete the akin exponentiation.
  - Includes the ability to read a single forward slash (/) as a divisive operator and completes the division, taking the first numerical value as the dividend and the second numerical value as the divisor.
  - Includes the ability to read the percent sign (%) as a modulo marker, taking the mod of the first numerical value, with respect to the second value.
  - Includes the ability to read and handle pairs of parentheses as grouping operators, not as actual arithmetic operators.

3.3 Supplementary Requirements

- The system shall provide a user-friendly and easy-to-read interface.
- The system shall clearly display the evaluated result.
- The system shall handle all errors gracefully, including invalid expressions and runtime errors.
- The system shall allow the user to retry input after an error occurs.

- The system shall maintain usability and clarity throughout user interaction.

4. Classification of Functional Requirements

Functionality	Type
Must parse arithmetic expressions that contain parentheses and enforces the correct operator precedence (basically needs to follow PEMDAS).	Essential
Has to evaluate standard operators (such as +, -, *, /, and %) using left-to-right associativity.	Essential
Must evaluate exponentiation operator (**) using right-to-left associativity.	Essential
Will process the unary operators (unary + and unary -).	Essential
Will read/calculate numeric constants (integers are assumed) within user input.	Essential
Easy to read command-line interface for entering expressions and viewing results.	Essential
Catching invalid inputs/expressions that are entered by the user. Catching undefined math operations (e.g. divide by zero) and displaying associated error messages.	Essential
Floating-point numeric inputs are allowed/accommodated by the program.	Desirable
Additional optimizations and features beyond essential functions and project scope.	Optional
Support for brackets [] and braces {} as additional grouping mechanisms alongside parentheses.	Optional

5. Appendices